

طراحی کامپیوتری سیستم‌های دیجیتال

Computer-Aided Digital System Design

مدارهای مجتمع قابل برنامه‌ریزی

1

1

چشم‌انداز

انواع مدارهای مجتمع قابل برنامه‌ریزی و ویژگی آن‌ها

سوییچ‌های قابل برنامه‌ریزی SRAM و آنتی‌فیوز

ساختار CPLD و FPGA و سلول‌های منطقی قابل برنامه‌ریزی

روش طراحی سیستم‌های دیجیتال با FPGA و VHDL

2

2

سیستم های تعبیه شده

سیستم تعبیه شده به این معناست که پروسسور (پردازنده) و سایر امکانات در یک سیستم واحد گنجانده شده اند.

3

3

روش های پیاده سازی سخت افزار در سیستم های تعبیه شده

- استفاده از تراشه *ASIC*
- استفاده از قطعات گسسته
- استفاده از کامپیوترهای صنعتی
- استفاده از تراشه های برنامه پذیر از قبیل *CPLD* و *FPGA*
- استفاده از پردازنده ها

4

4

روش های پیاده سازی سخت افزار در سیستم های تعبیه شده

استفاده از تراشه ASIC

- ASIC مخفف *Application Specific Integrated Circuit* است.
- در این روش کلیه نیازهای سیستم در قالب یک مدار مجتمع، توسط تکنولوژی VLSI و به صورت بهینه طراحی می شود.



فقط در سیستم هایی با تولید بالا مانند گوشی های تلفن همراه یا کاربردهای نظامی که کارآیی و قابلیت اطمینان اهمیت دارد، استفاده می شود.



- هزینه اولیه طراحی مهندسی بالا
- طی نمودن پروسه زمان بر و پرهزینه
- طراحی تراشه در زمان ارتقای سیستم

5

5

روش های پیاده سازی سخت افزار در سیستم های تعبیه شده

استفاده از قطعات گسسته

- طراحی سیستم توسط المان هایی مانند ترانزیستورها و تراشه هایی مانند سری 74XX انجام می شود.



- به دلیل تعداد المان های زیاد مورد استفاده، ابعاد و صرف توان سیستم بسیار افزایش می یابد

امروزه به ندرت از این روش استفاده می شود



6

6

روش های پیاده سازی سخت افزار در سیستم های تعبیه شده

استفاده از کامپیوترهای صنعتی

- در برخی از کاربردها مانند دستگاه های خودپرداز *ATM* می توان از کامپیوترهای صنعتی به عنوان پردازنده استفاده کرد.



- سخت افزار سیستم به صورت آماده بوده و نیازی به طراحی ندارد.
- سیستم عامل های موجود از قبیل *Linux* و *Windows CE* کلیه سرویس های اصلی مورد نیاز سیستم را تأمین می کند.
- می توان از بسته های نرم افزاری متعددی برای نوشتن برنامه استفاده نمود.

7

7

روش های پیاده سازی سخت افزار در سیستم های تعبیه شده

استفاده از کامپیوترهای صنعتی



- ابعاد بزرگ و مصرف توان بالای سیستم باعث می شود که در بسیاری از سیستم های کم مصرف و قابل حمل نتوان از این روش استفاده کرد.
- سیستم عامل های معمول مانند ویندوز قابلیت اطمینان بالایی ندارند و نمی توان در کاربردهای حساس از آن ها استفاده کرد.
- به صورت بلادرنگ (*Real Time*) عمل نمی کنند که همین امر، کاربرد آنها را محدود می کند.

8

8

روش های پیاده سازی سخت افزار در سیستم های تعبیه شده

استفاده از تراشه های برنامه پذیر از قبیل CPLD و FPGA

- این ادوات در مقایسه با تراشه های ASIC هزینه طراحی اولیه کمتری دارند.
- کارد اصلی تراشه های FPGA در سیستم های پردازش اطلاعات موازی و پرسرعت است.
- از تراشه های CPLD بیشتر به عنوان کنترل کننده حافظه استفاده می شود.



- مصرف توان تراشه های FPGA بیش از میکروکنترلرها است



کمتر به عنوان کنترل کننده اصلی
سیستم در ادوات تعبیه شده
استفاده می شوند.

9

9

روش های پیاده سازی سخت افزار در سیستم های تعبیه شده

استفاده از پردازنده ها

میکروپروسسورهای عمومی (همه منظوره)

پردازنده های DSP

میکروکنترلرها

با توجه به نوع کاربرد

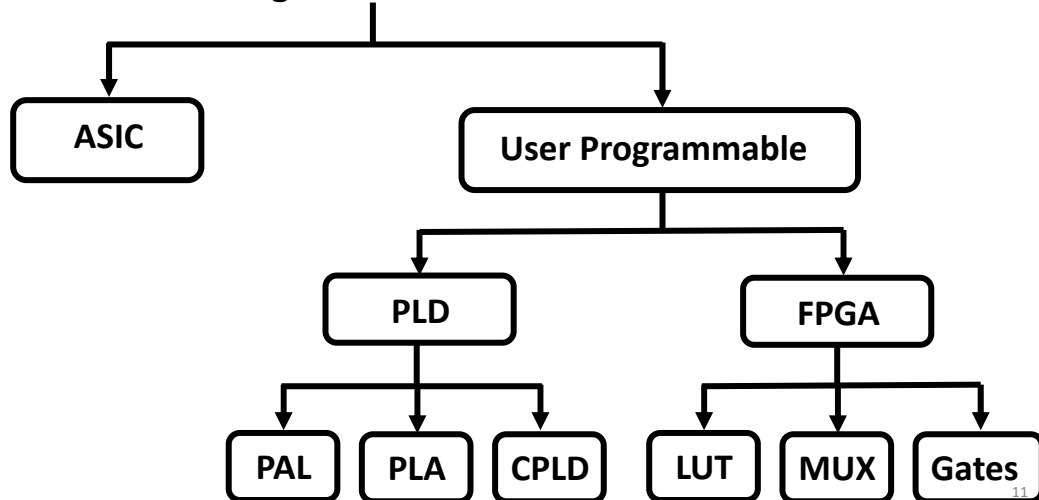
10

10

روش های پیاده سازی سخت افزار در سیستم های تعبیه شده

مدارهای مجتمع

Integrated Circuits



11

انواع مدارهای منطقی برنامه پذیر

- در گذشته، طراحی سیستم های دیجیتال با مجموعه ای از گیت ها و IC های استاندارد انجام می شد.
- امروزه طراحی این سیستم ها با استفاده از مدارهای منطقی برنامه پذیر تغییرات زیادی نموده است.
- این مدارها با ظرفیتی در حدود ۱۰۰ تا چند هزار گیت در یک مدار مجتمع VLSI قرار دارند.

Field Programmable Device (FPD)
Programmable Logic Device (PLD)



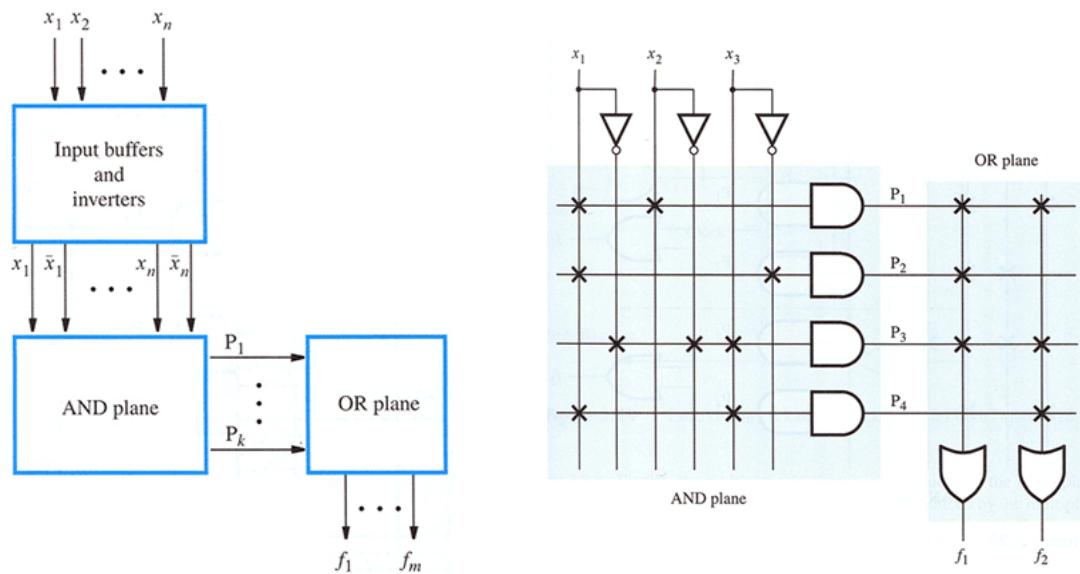
مدارهای منطقی برنامه پذیر

Very Large Scale Integrated Circuits (VLSI)

12

12

مدارهای منطقی برنامه‌پذیر PLA

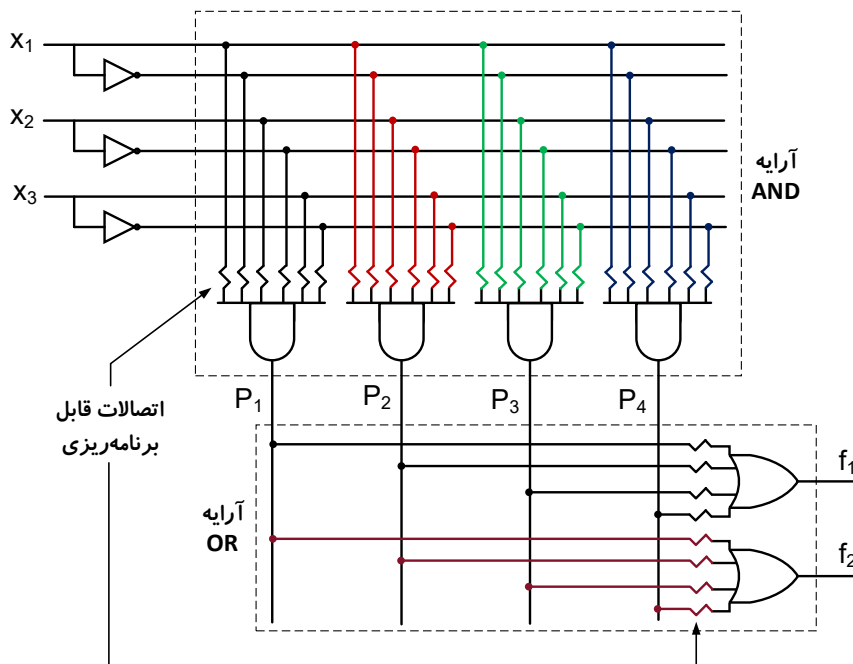


PLA = Programmable Logic Array

13

13

مدارهای منطقی برنامه‌پذیر PLA

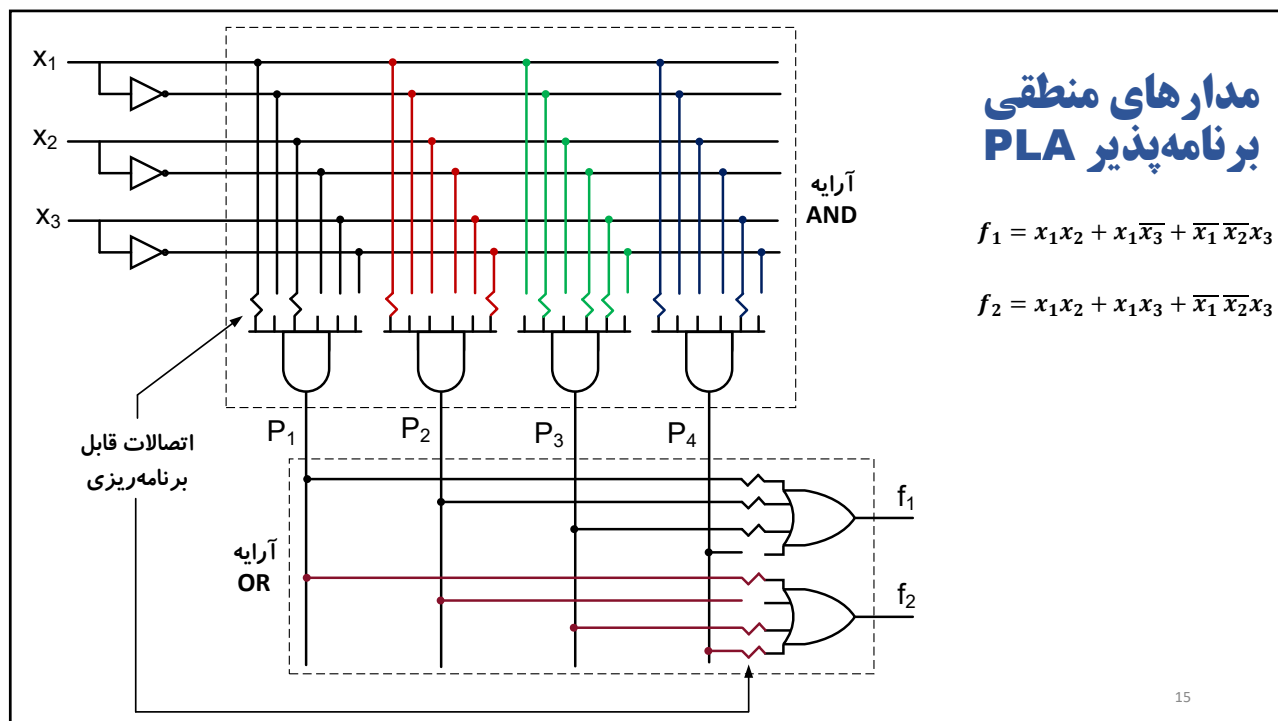


$$f_1 = x_1x_2 + x_1\bar{x}_3 + \bar{x}_1\bar{x}_2x_3$$

$$f_2 = x_1x_2 + x_1x_3 + \bar{x}_1\bar{x}_2x_3$$

14

14



15

مدارهای منطقی برنامه‌پذیر PLA

• PLA شامل یک طبقه آرایه‌ای از گیت AND و یک طبقه آرایه از گیت OR می‌باشد.

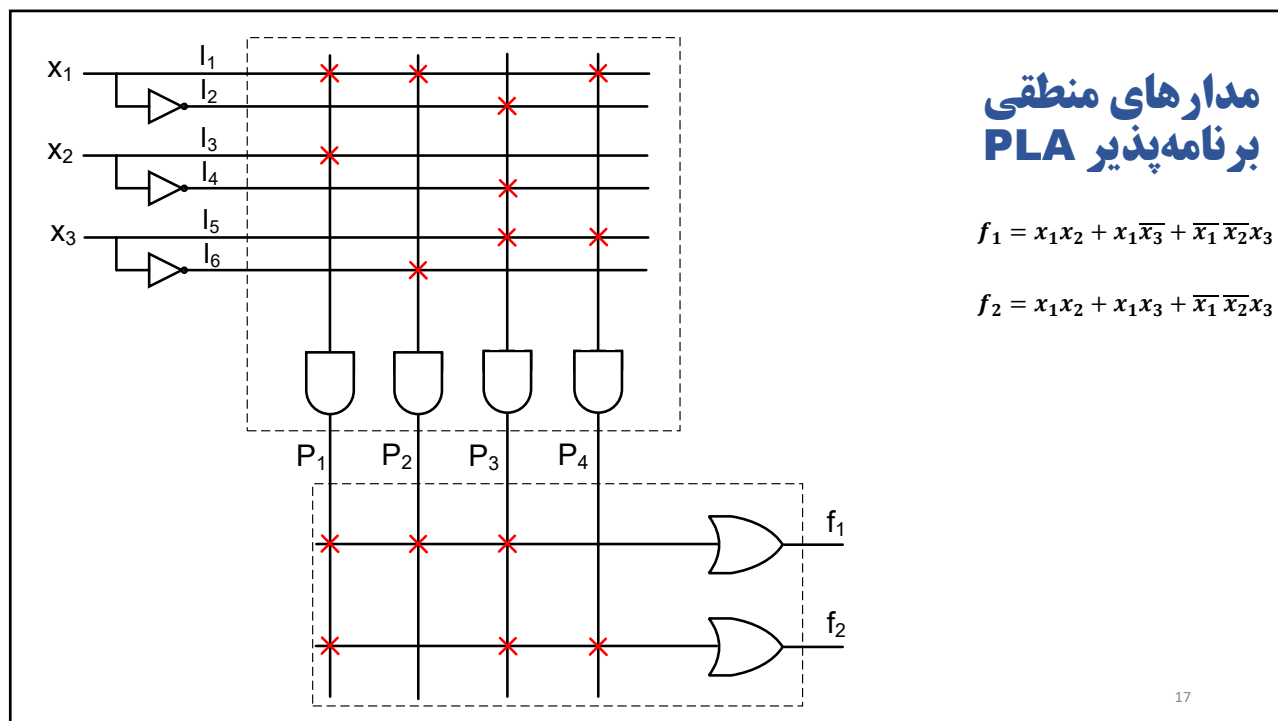
• هر یک از ورودی‌ها یا معکوس آن‌ها به گیت AND متصل می‌شوند، در نتیجه خروجی گیت AND حاصلضرب ورودی یا معکوس آن‌ها است.

• خروجی هر گیت OR، مجموع منطقی خروجی‌های AND می‌باشد.

• مدار PLA برای پیاده‌سازی توابع منطقی که به صورت مجموع حاصلضرب (SOP) است بسیار مناسب می‌باشد.

16

16



17

آیا هر تابع منطقی را می‌توان با PLA پیاده‌سازی کرد؟

چون هر تابع منطقی را می‌توان برحسب مجموع حاصل ضرب (SOP) نوشت،
لذا هر تابع منطقی قابل پیاده‌سازی بر PLA است

18

18

مدارهای منطقی برنامه‌پذیر PLA

• در آرایه PLA هر خط عمودی داخل آرایه AND، خروجی یک مدار AND است که در محل تقاطع هر سطر و ستون عمودی یک **سوییچ قابل برنامه‌ریزی** گذاشته شده است.

• سوییچ قابل برنامه‌ریزی یک **دیود با فیوز به‌طور سری** یا یک **ترانزیستور MOS با گیت شناور** باشد.

• این سوییچ‌های قابل برنامه‌ریزی با ابزارهای برنامه‌ریزی CAD برنامه‌ریزی می‌شوند.

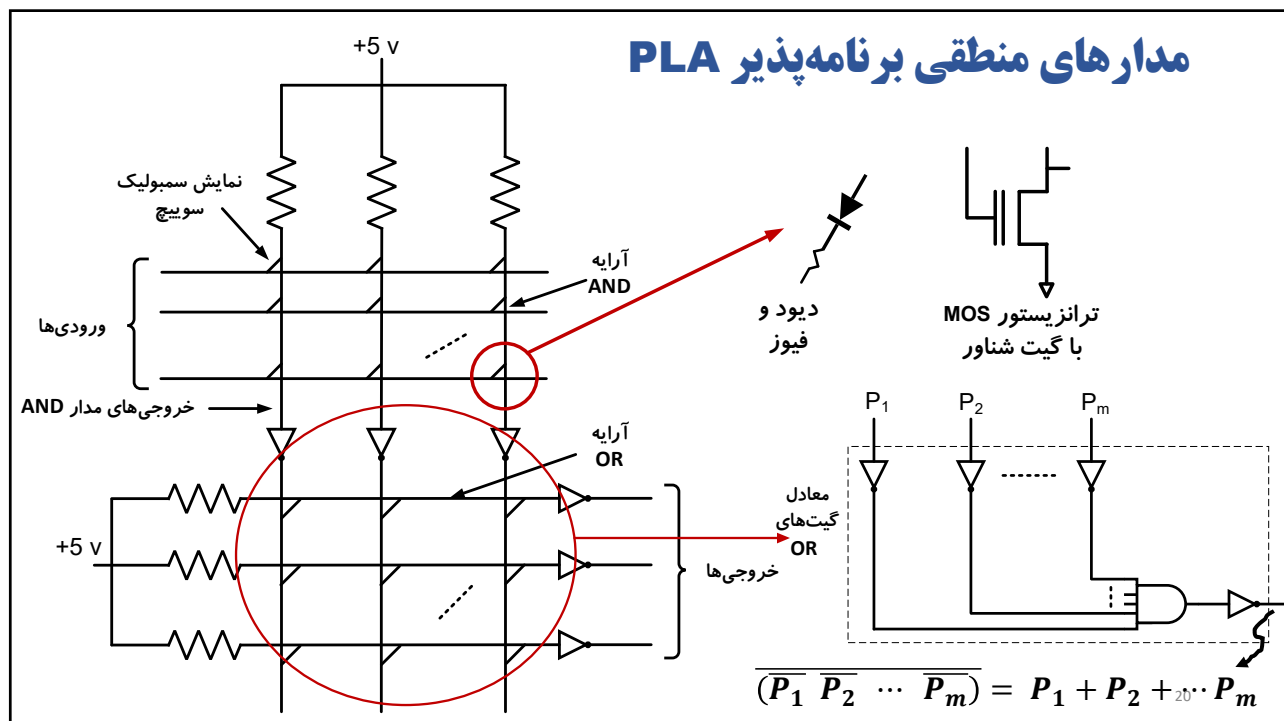
Computer Aided Design (CAD)

Programmable Switch = سوییچ قابل برنامه‌ریزی

Floating Gate = گیت شناور

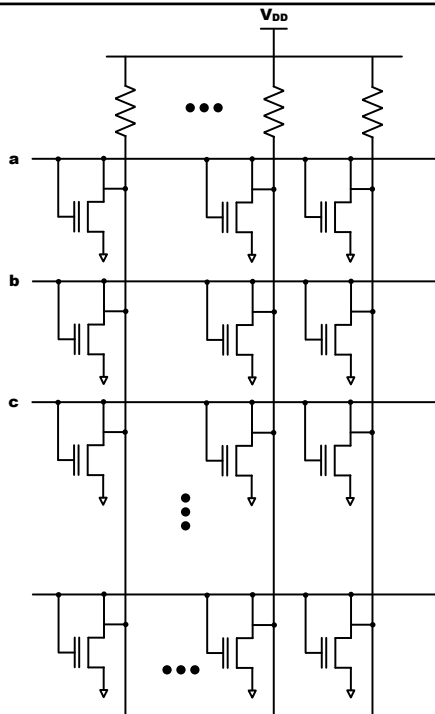
19

19



20

مدارهای منطقی برنامه‌پذیر PLA



21

21

مدارهای منطقی برنامه‌پذیر PLA

• چون PLA دو طبقه گیت قابل برنامه‌ریزی دارد **گران** تمام می‌شود و **تأخیر** مدار نسبتاً **زیاد** است، در نتیجه **سرعت آن کم** است.

• برای حل این مسأله از مدارهای قابل برنامه‌ریزی **PAL** استفاده می‌شود.

Programmable Array Logic (PAL)

22

22

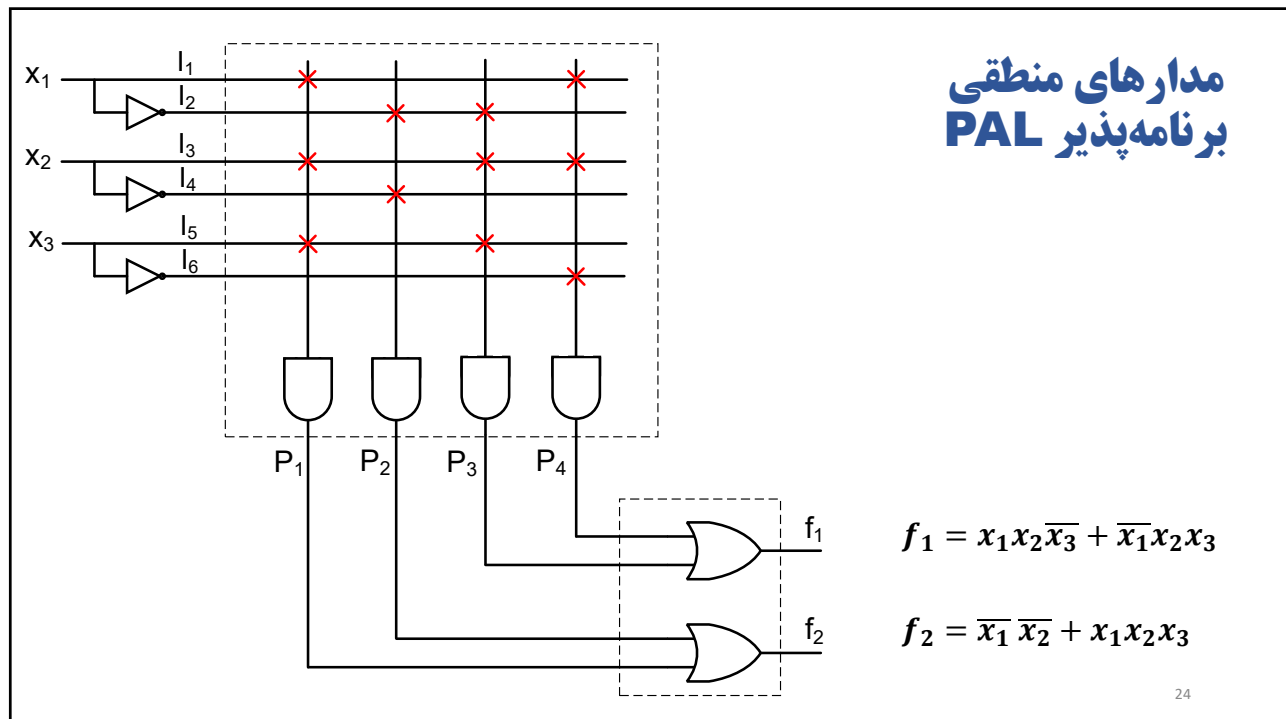
مدارهای منطقی برنامه‌پذیر PAL

• در PLA هر دو آرایه AND و OR قابل برنامه‌ریزی هستند.

• در PAL فقط آرایه AND قابل برنامه‌ریزی است و خروجی گیت‌های AND به‌طور دائم به ورودی‌های OR متصل می‌باشند.

23

23



24

مدارهای منطقی برنامه‌پذیر PAL

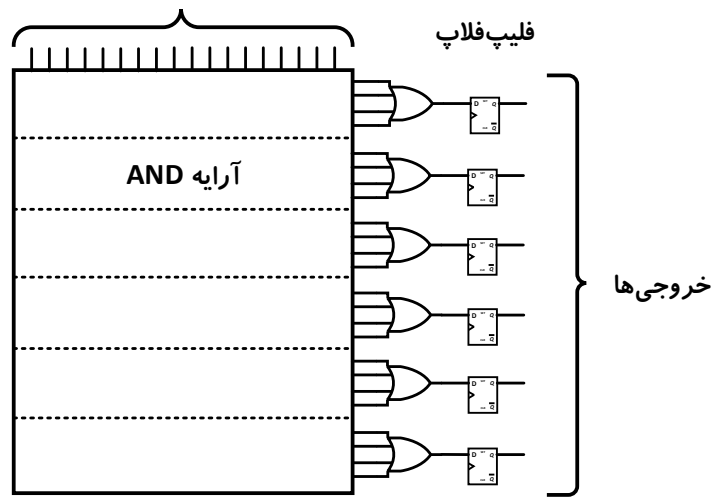
- در PAL چون فقط گیت‌های AND قابل برنامه‌ریزی هستند، لذا ساخت PAL کم‌هزینه‌تر از PLA می‌باشد و سرعت آن نیز بیشتر است.
- برای جبران آرایه غیرقابل برنامه‌ریزی OR، سازندگان انواع مختلفی از PAL را به بازار عرضه کرده‌اند که از لحاظ تعداد گیت‌های OR و نیز تعداد ورودی‌های گیت‌های OR با یکدیگر متفاوت هستند.
- در PAL‌ها معمولاً خروجی گیت‌های OR به فلیپ‌فلاپ متصل هستند، پس می‌توان مدارهای ترتیبی با پالس ساعت را در آن‌ها پیاده‌سازی نمود.

25

25

مدارهای منطقی برنامه‌پذیر PAL

ورودی‌ها و فیدبک فلیپ‌فلاپ‌ها



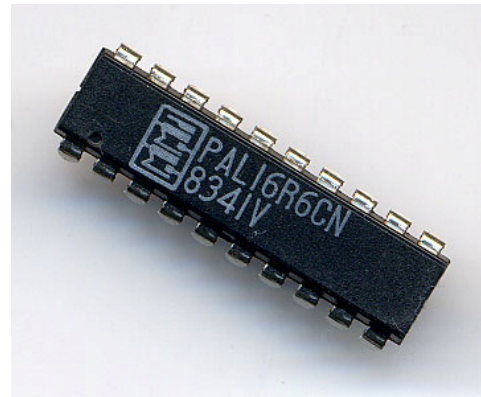
26

26

مدارهای منطقی برنامه‌پذیر PAL

PAL16R6

PAL16R8



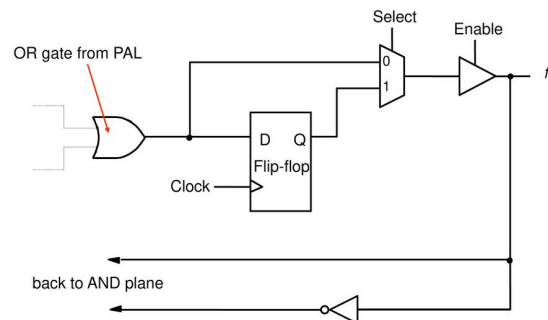
27

27

مدارهای منطقی برنامه‌پذیر PAL

• در مدارهای PAL نسل بعدی به جای فلیپ‌فلاپ از مجموعه‌ای از فلیپ‌فلاپ، مولتی‌پلکسر و ... به نام ماکروسل (Macrocell) استفاده شد.

• با برنامه‌ریزی ورودی مولتی‌پلکسر می‌توان خروجی PAL را مستقیماً برابر تابع f یا معکوس آن استفاده نمود و یا خروجی تابع را از فلیپ‌فلاپ گرفت.



28

28

مدارهای منطقی برنامه‌پذیر PAL

- امروزه PAL‌های مختلف با انواع ماکروسل‌ها در بازار وجود دارند که نمونه‌ای از آن‌ها PAL یا PLD به شماره **PAL22V10** است

- مدارهای منطقی برنامه‌پذیر PAL و PLA به **مدارهای برنامه‌پذیر ساده (SPLD)** معروف هستند و ظرفیت آن‌ها حدود **۲۰۰ گیت** می‌باشد.

- SPLD‌ها برای محدوده وسیعی از کاربردها مناسب هستند و امروزه تقریباً در هر سیستم دیجیتال وجود دارند.

Programmable Logic Device (PLD)

Simple Programmable Logic Device (SPLD)

29

29

مدارهای منطقی برنامه‌پذیر CPLD

- PAL‌ها، IC‌های قابل برنامه‌ریزی خوبی هستند اما برای طراحی سیستم‌های پیچیده دیجیتال ممکن است چندین PAL نیاز باشد.

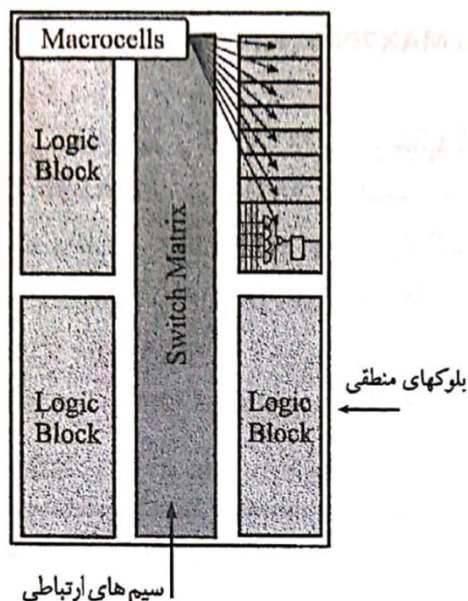
- مدارهای منطقی CPLD از دو یا چندین **بلوک منطقی** و تعدادی **ماکروسل** تشکیل شده‌اند که با **سیستم‌های ارتباطی** و **سوییچ‌های قابل برنامه‌ریزی** با هم ارتباط داده می‌شوند.

Switch Matrix = سوییچ‌های قابل برنامه‌ریزی
Complex Programmable Logic Device (CPLD)

30

30

مدارهای منطقی برنامه‌پذیر CPLD



31

31

مدارهای منطقی برنامه‌پذیر CPLD

• CPLDهای تجاری با اندازه‌های مختلف از ۲ تا ۱۰۰ بلوک منطقی یا SPLD و با ظرفیتی در حدود ۱۰۰۰ تا ۱۵۰۰۰ گیت ساخته شده‌اند.

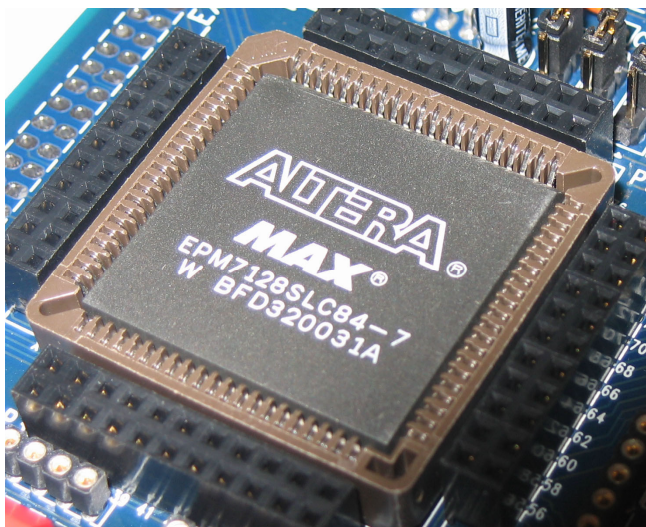
• CPLD خانواده MAX9000 از شرکت Altera دارای ظرفیت در حدود ۱۲۰۰۰ گیت می‌باشد.



32

32

مدارهای منطقی برنامه‌پذیر CPLD



سری‌های CPLD شرکت Altera

MAX5000

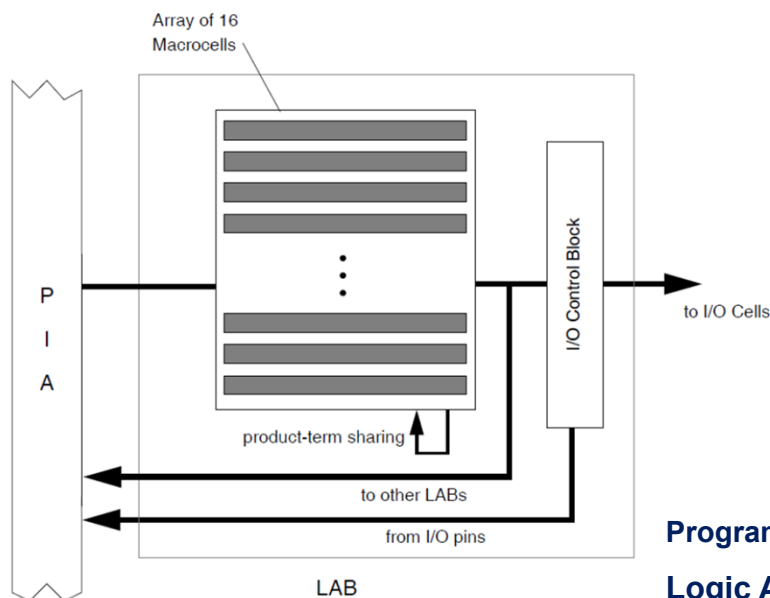
MAX7000

MAX9000

33

33

مدارهای منطقی برنامه‌پذیر CPLD



MAX7000 سری CPLD

Programmable Interconnect Array (PIA)

Logic Array Block (LAB)

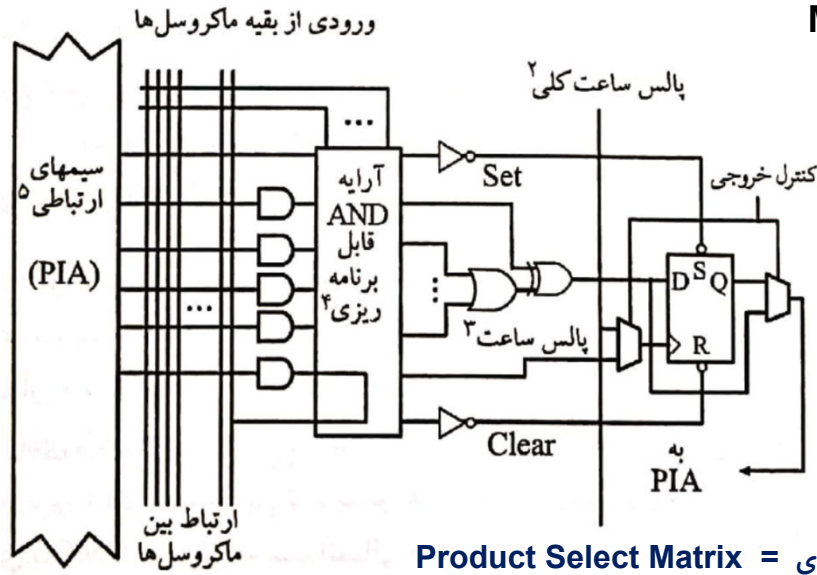
34

34

مدارهای منطقی برنامه‌پذیر CPLD

CPLD سری MAX7000

Programmable Interconnect Array (PIA)
پالس ساعت = Array Clock



آرایه AND قابل برنامه ریزی = Product Select Matrix

35

35

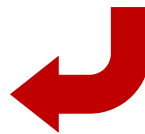
مدارهای منطقی برنامه‌پذیر CPLD

CPLD های شرکت Xilinx

XC7000

XC9500

مشابه سری MAX7000 شرکت Altera



36

36

مزایای مدارهای منطقی برنامه‌پذیر CPLD

- CPLD تأخیر کمی در حد چند نانوثانیه دارند، لذا بسیار سریع و حدود فرکانس ۱۰۰ مگاهرتز کار می‌کنند.
- CPLD ها با ابزارهای برنامه‌ریزی CAD که توسط سازندگان تهیه شده است قابل برنامه‌ریزی هستند.
- CPLD به علت سرعت زیاد و ظرفیت بالا (حدود چند هزار گیت در یک IC) برای طراحی نمونه‌سازی سیستم دیجیتال، کنترل‌کننده‌های گرافیکی، شبکه‌های کامپیوتری LAN، قسمت‌های مختلف پروسور و ... به کار برده می‌شوند.

37

37

مزایای مدارهای منطقی برنامه‌پذیر CPLD

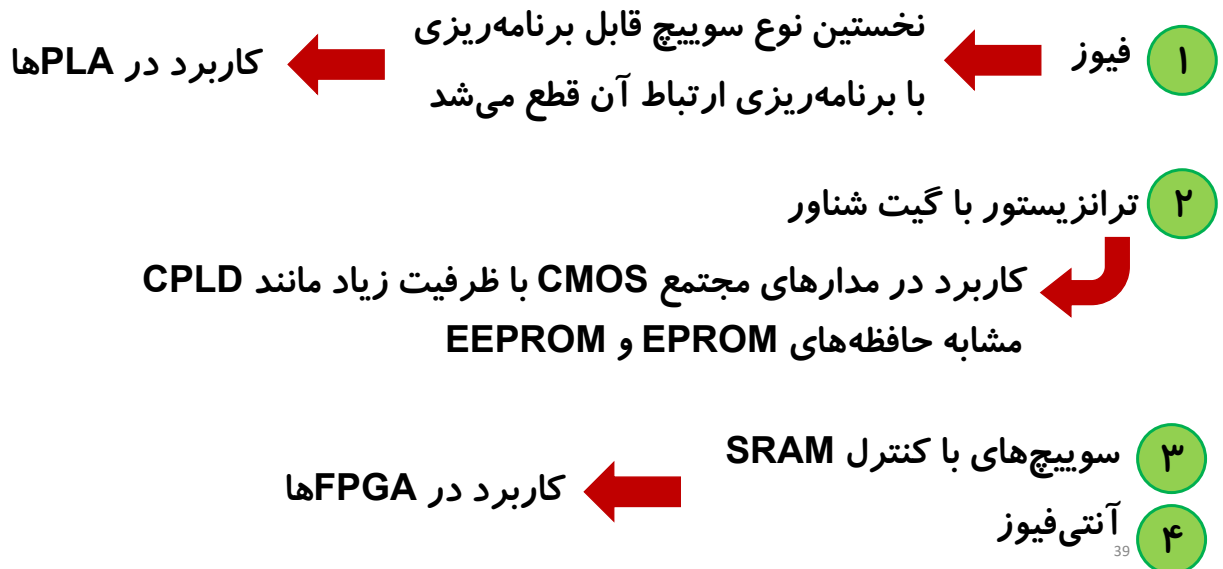
به‌طور کلی CPLD برای طراحی سیستم‌های دیجیتالی که به تعداد زیاد گیت‌های AND و OR نیاز دارند به کار برده می‌شود.

یکی از محاسن CPLD این است که قابل برنامه‌ریزی مجدد می‌باشد.

38

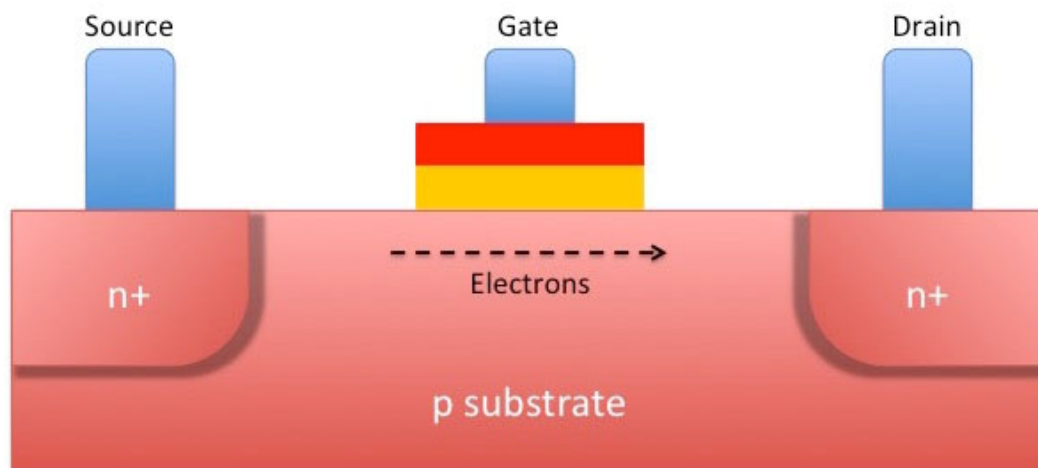
38

سوییچ‌های قابل برنامه‌ریزی CPLD و PPGA



39

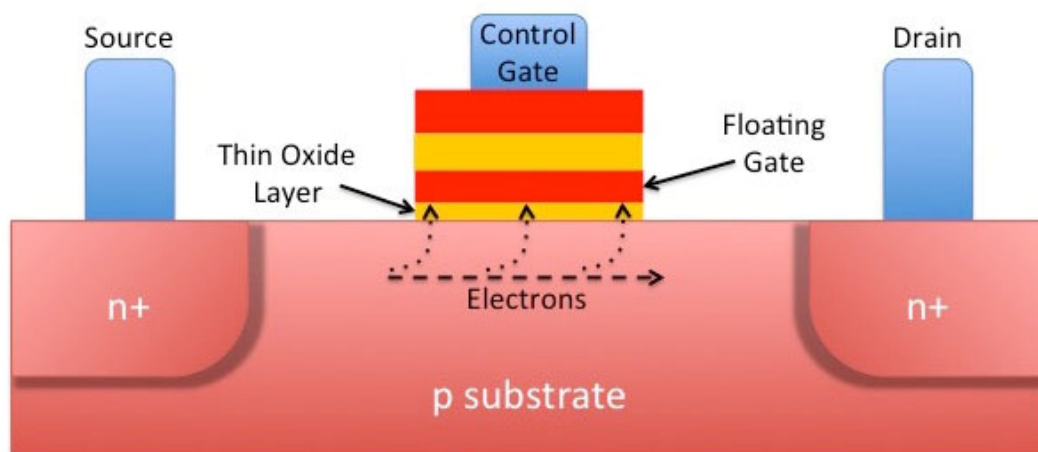
ترانزیستور سوییچ قابل برنامه‌ریزی با گیت شناور



40

40

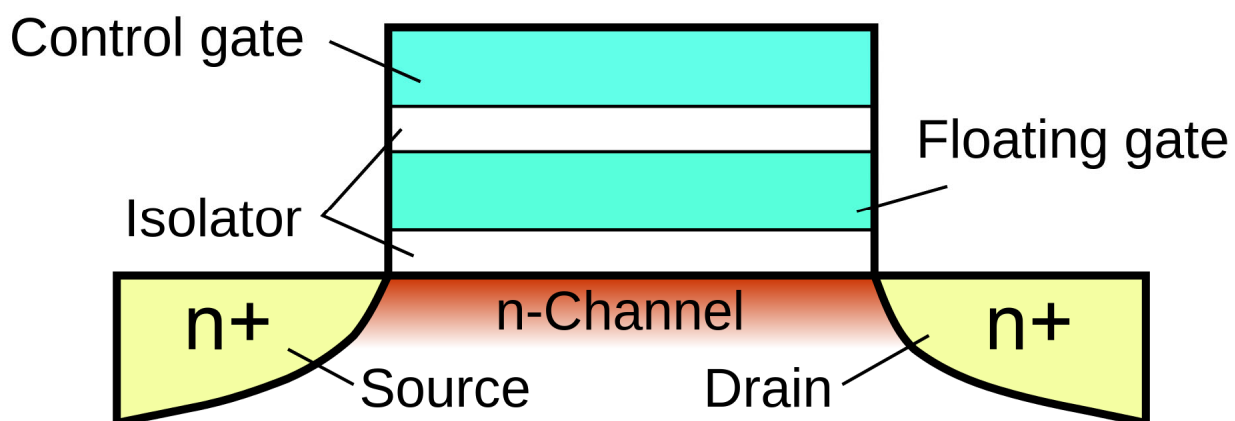
ترانزیستور سویچ قابل برنامه‌ریزی با گیت شناور



41

41

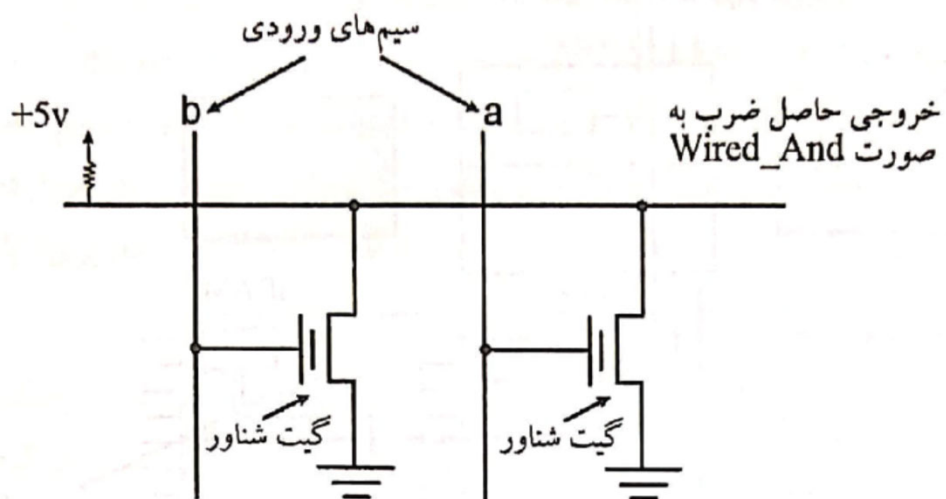
ترانزیستور سویچ قابل برنامه‌ریزی با گیت شناور



42

42

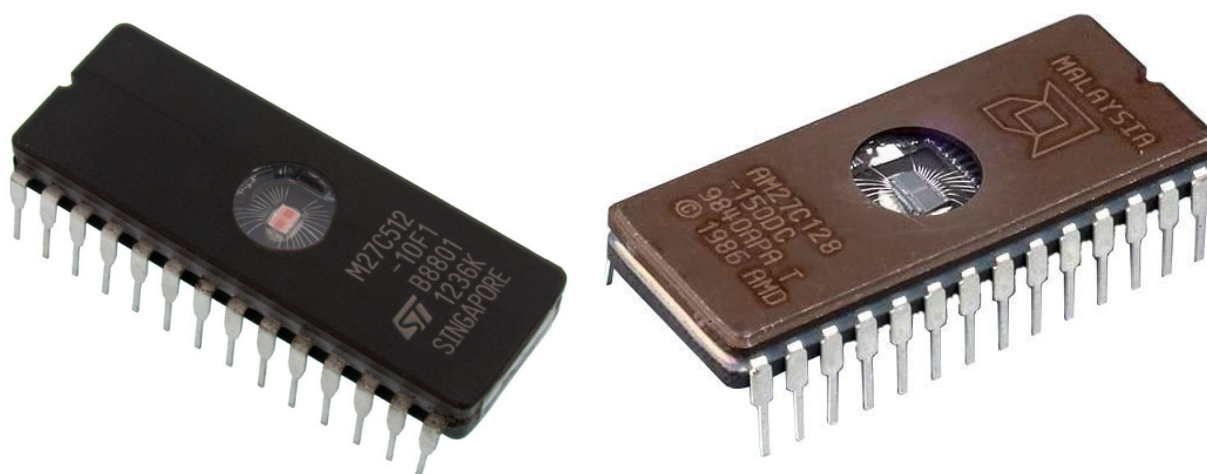
ترانزیستور سویچ قابل برنامه‌ریزی با گیت شناور



43

43

ترانزیستور سویچ قابل برنامه‌ریزی با گیت شناور (EPROM)



44

44

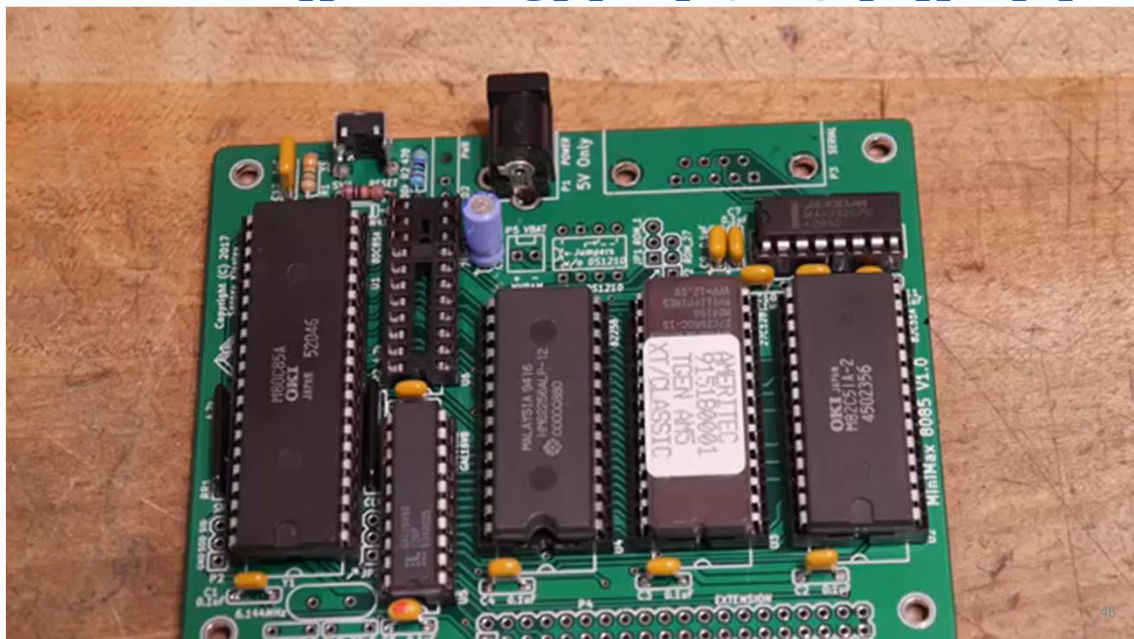
ترانزیستور سویچ قابل برنامه‌ریزی با گیت شناور (EPROM)



45

45

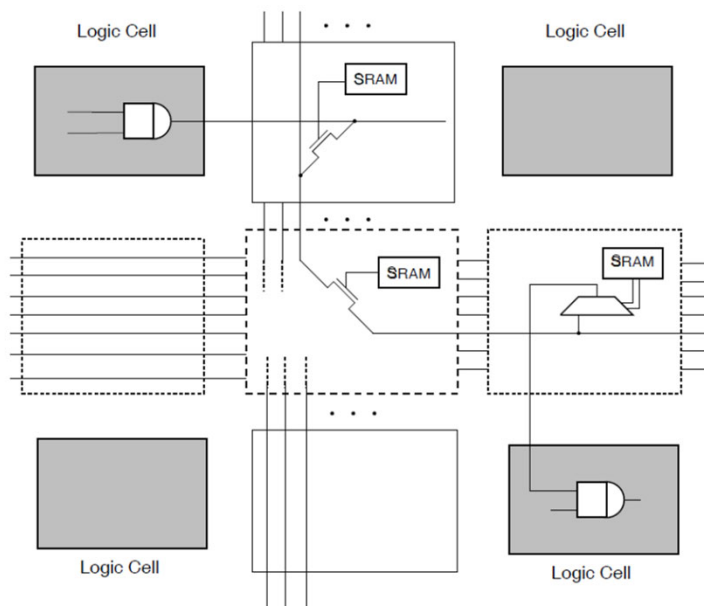
ترانزیستور سویچ قابل برنامه‌ریزی با گیت شناور (EPROM)



46

46

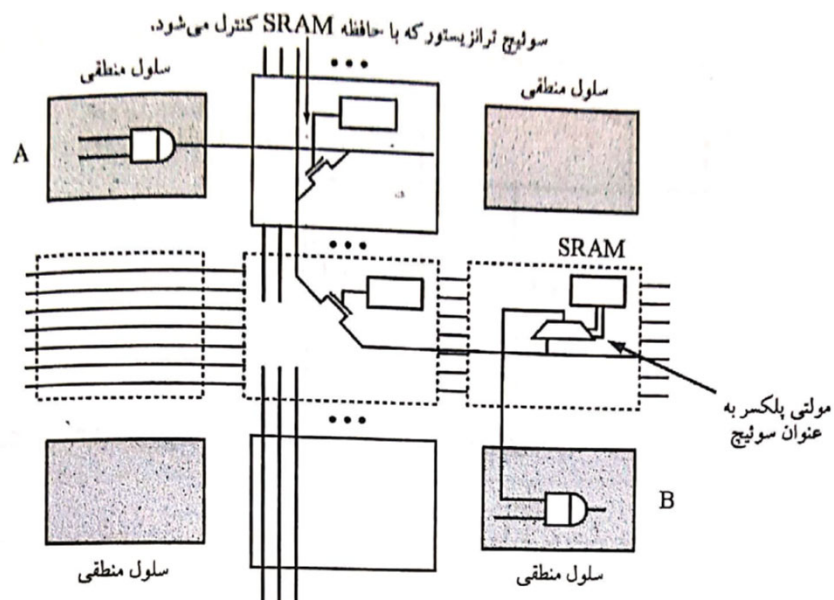
سوئیچ قابل برنامه‌ریزی با حافظه SRAM



47

47

سوئیچ قابل برنامه‌ریزی با حافظه SRAM



48

48

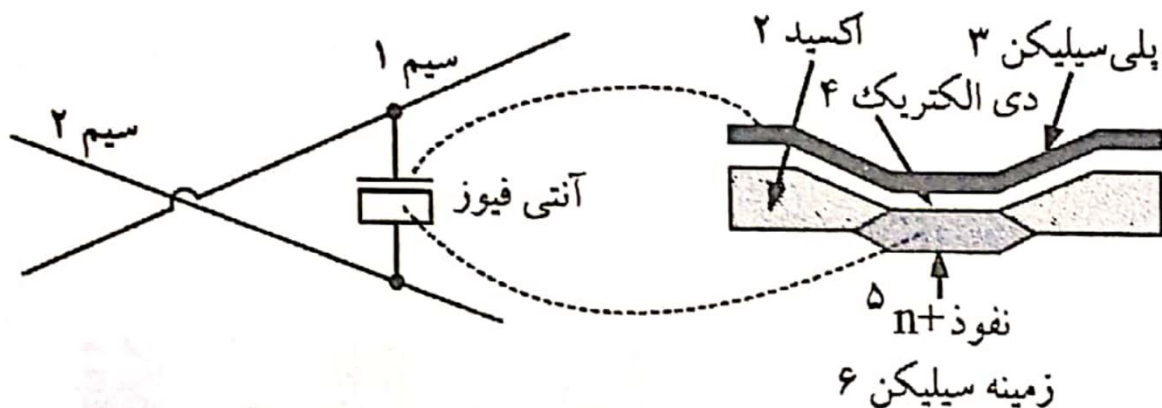
سوییچ قابل برنامه‌ریزی با حافظه SRAM

- در برخی از FPGAها، سوییچ‌های قابل برنامه‌ریزی ترانزیستوری به کار برده می‌شود که با حافظه SRAM برنامه‌ریزی یا کنترل می‌شوند.
- اگر خروجی حافظه SRAM برابر با ۱ شود، سوییچ ترانزیستوری بسته شده و دو سیم را به هم متصل می‌کند.
- در برخی از FPGAها از مولتی‌پلکسر به عنوان سوییچ استفاده می‌شود که با قرار دادن مقدار حافظه SRAM در قسمت کنترل مولتی‌پلکسر، هر یک از ورودی‌های مورد نظر مولتی‌پلکسر به خروجی آن متصل می‌شود.

49

49

سوییچ قابل برنامه‌ریزی آنتی‌فیوز



50

50

سویچ قابل برنامه‌ریزی آنتی‌فیوز

- سویچ آنتی‌فیوز فقط یک بار برنامه‌ریزی می‌شود لذا FPGA برای همیشه برنامه‌ریزی می‌گردد و قابل برنامه‌ریزی مجدد نمی‌باشد.
- سویچ SRAM با تغییر مقدار SRAM مجدداً برنامه‌ریزی می‌شود، لذا FPGA‌های با سویچ SRAM را می‌توان مجدداً برنامه‌ریزی نمود.
- اشکال سویچ SRAM این است که با قطع برق اطلاعات آن از بین می‌رود لذا باید باتری یا وسایل دیگر برای حالت قطع برق در مدار FPGA قرار داده شود.

51

51

مقایسه سویچ‌های قابل برنامه‌ریزی

نام	قابلیت برنامه‌ریزی مجدد	فرار بودن	فن‌آوری ساخت
فیوز	خیر	خیر	Bipolar
EPROM	بله خارج از مدار	خیر	UVC MOS
EEPROM	بله درون مدار	خیر	EECMOS
SRAM	بله درون مدار	بله	CMOS
آنتی‌فیوز	خیر	خیر	CMOS+

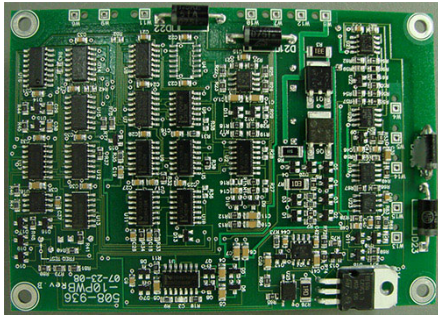
52

52

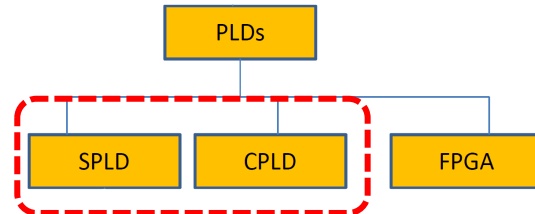
چرا FPGA؟

چند نوع پیاده سازی سخت افزاری داریم؟

با قطعات گسسته و آماده



با سخت افزارهای برنامه پذیر



53

53

چرا FPGA؟

چند نوع پیاده سازی سخت افزاری داریم؟

با قطعات گسسته و آماده

- ✓ تعداد قطعات زیاد
- ✓ غیر قابل تغییر
- ✓ حساس به نویز
- ✓ قطعات محدود
- ✓ طراحی زمان بر
- ✓
- ✓

با سخت افزارهای برنامه پذیر

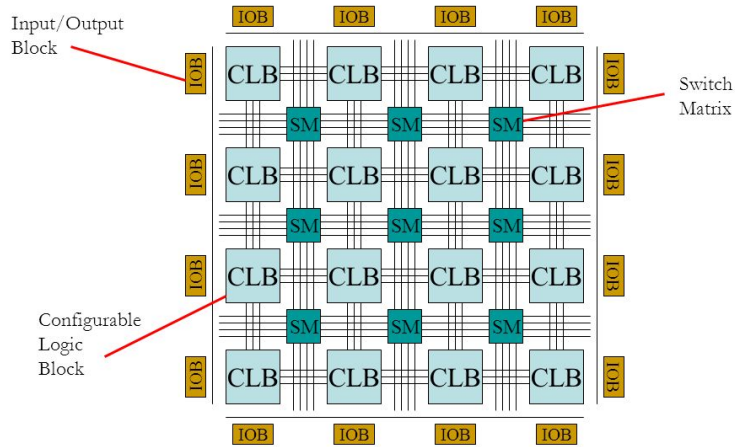
- ✓ سریع :FPGA
- ✓ کم توان
- ✓ قابل تغییر
- ✓ مناسب برای طراحی های پیچیده
- ✓ موجود در اندازه های کوچک
- ✓
- ✓ برای طراحی های ساده

54

54

به شکل کلی چند نوع FPGA داریم؟

❖ یک FPGA از چه چیزی ساخته شده؟

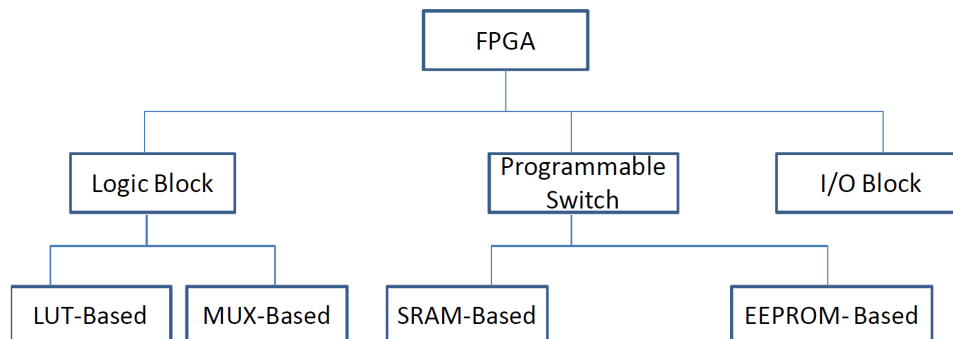


55

55

به شکل کلی چند نوع FPGA داریم؟

❖ یک FPGA از چه چیزی ساخته شده؟



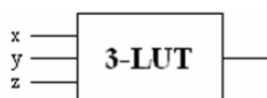
56

56

به شکل کلی چند نوع FPGA داریم؟

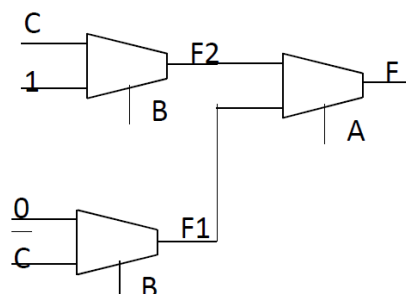
➤ Logic Block ها چند نوع هستند؟

LUT



x	y	z	f
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

MUX



57

57

به شکل کلی چند نوع FPGA داریم؟

➤ Logic Block ها چند نوع هستند؟

LUT

- ✓ فرآیند طراحی ساده تر است.
- ✓ پیچیدگی و تاخیر طراحی به پیچیدگی تابع وابسته نیست.
- برای ساخت، ترانزیستور زیاد نیاز دارد.

MUX

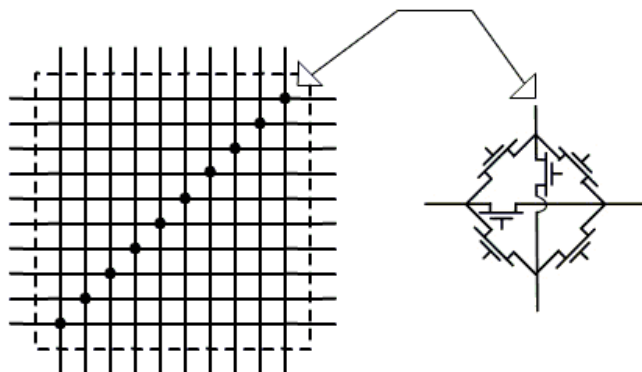
- ✓ تعداد ترانزیستور وابسته به طراحی است.
- تاخیر برای توابع پیچیده بیشتر می شود.
- روند طراحی پیچیده تر است.

58

58

به شکل کلی چند نوع FPGA داریم؟

➤ Switch ها چند نوع هستند؟

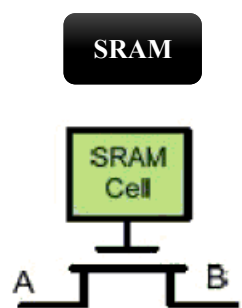


59

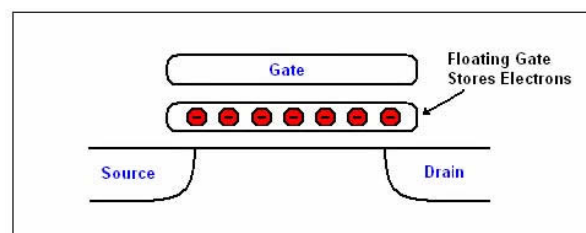
59

به شکل کلی چند نوع FPGA داریم؟

➤ Switch ها چند نوع هستند؟



EEPROM



60

60

به شکل کلی چند نوع FPGA داریم؟

➤ Switch ها چند نوع هستند؟

SRAM

✓ قابل برنامه ریزی مجدد.

➤ ترانزیستور زیادی نیاز دارد.

➤ حافظه آن فرار است.

EEPROM

✓ قابل برنامه ریزی مجدد.

✓ ترانزیستور کمتر می خواهد.

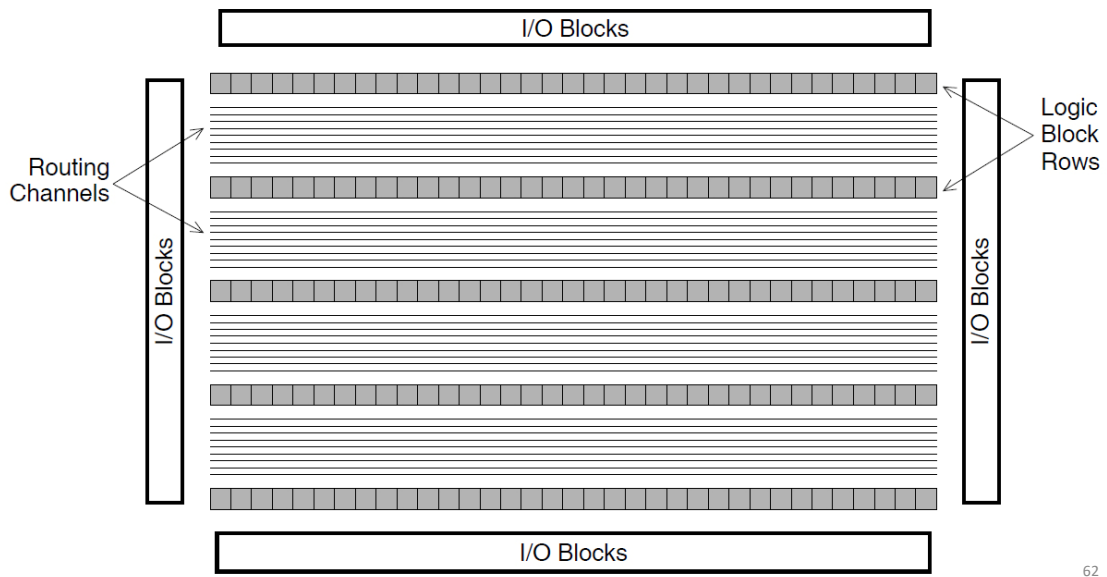
✓ حافظه آن غیر فرار است.

➤ گران تر است.

61

61

ساختار FPGA با سلول های منطقی به صورت ردیفی



62

62

دو اختلاف اساسی بین FPGA و CPLD

سرعت

در FPGA، سویچ‌های قابل برنامه‌ریزی زیادی برای اتصال بلوک‌های FPGA به کار می‌رود که دارای تأخیر می‌باشند.

FPGAها تأخیر بیشتری نسبت به CPLDها و PALها دارند.

امکانات

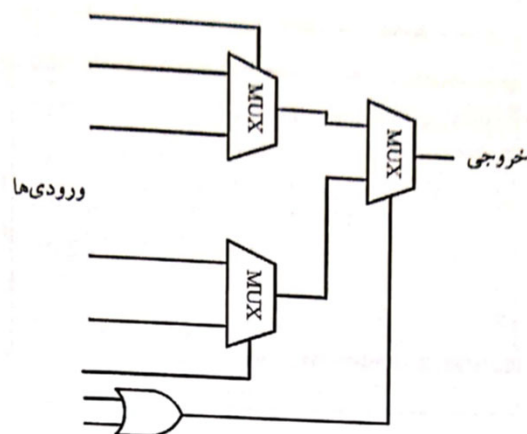
FPGAها با ظرفیت در حدود چند هزار گیت، دارای امکانات بیشتری نسبت به CPLDها می‌باشند.

می‌توان با استفاده از FPGAها مدارهای پیچیده‌تر و بزرگتری را طراحی نمود

63

63

سلول منطقی FPGA مبتنی بر MUX خانواده Act-1 از شرکت Actel



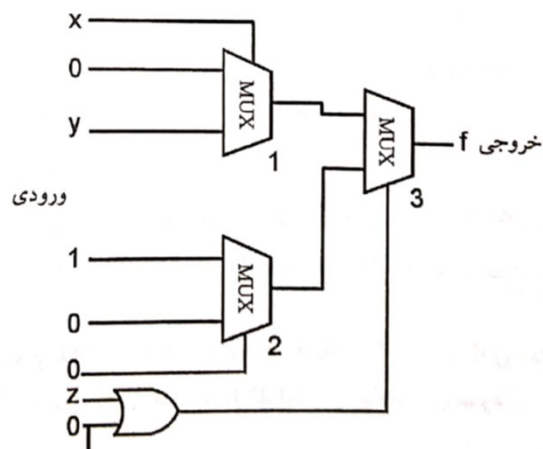
64

64

سلول منطقی FPGA مبتنی بر MUX خانواده Act-1 از شرکت Actel

x	y	z	f
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	0	1	1
1	1	0	1
1	1	1	1

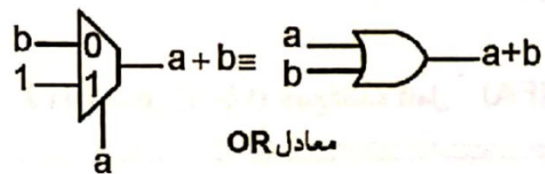
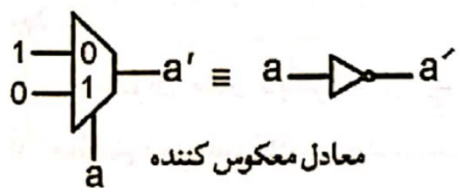
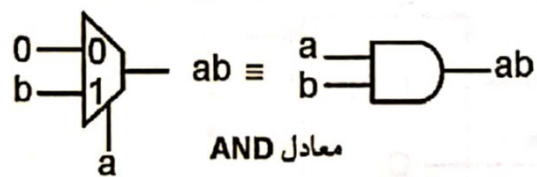
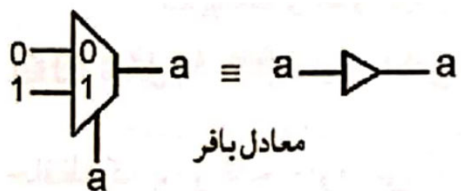
$$f = xy + z$$



65

65

سلول منطقی FPGA مبتنی بر MUX خانواده Act-1 از شرکت Actel



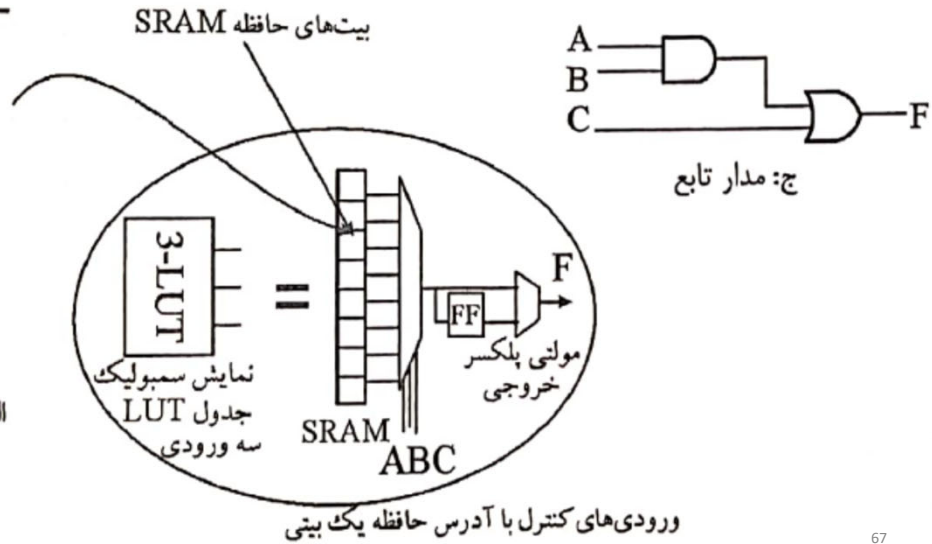
66

66

سلول منطقی FPGA مبتنی بر LUT

ABC	F
000	0
001	1
010	0
011	1
100	0
101	1
110	1
111	1

الف: جدول درستی تابع F
 $F=AB+C$

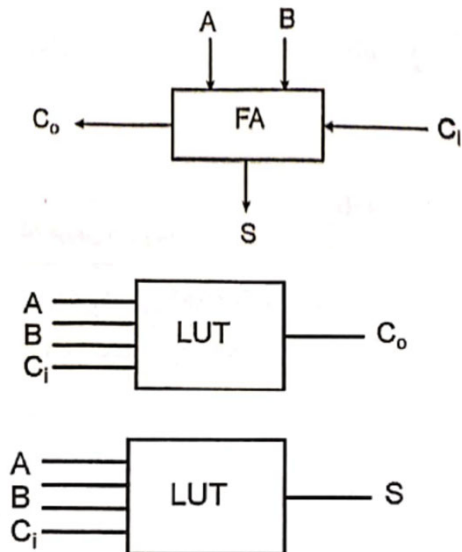


67

67

سلول منطقی FPGA مبتنی بر LUT

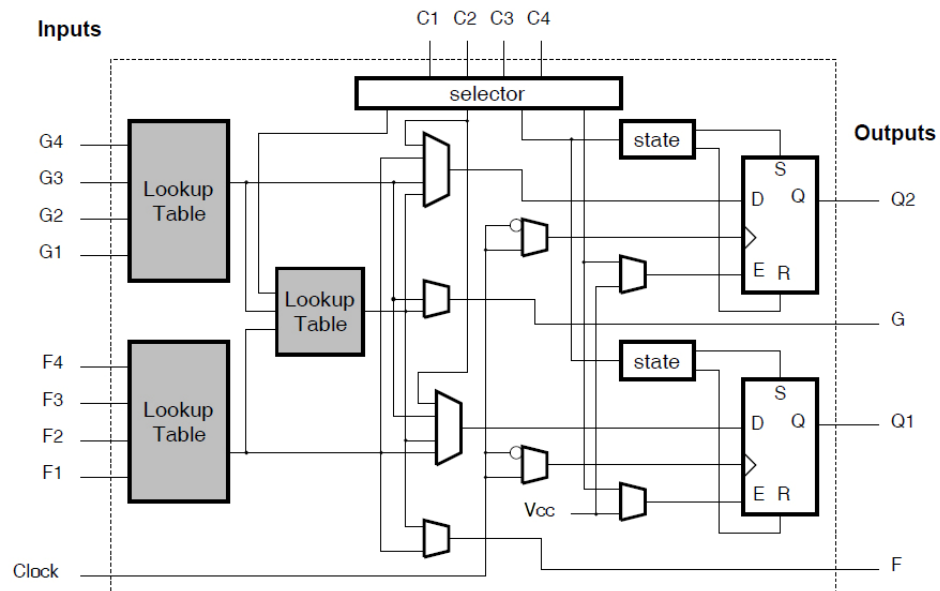
A	B	C_i	S	C_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



68

68

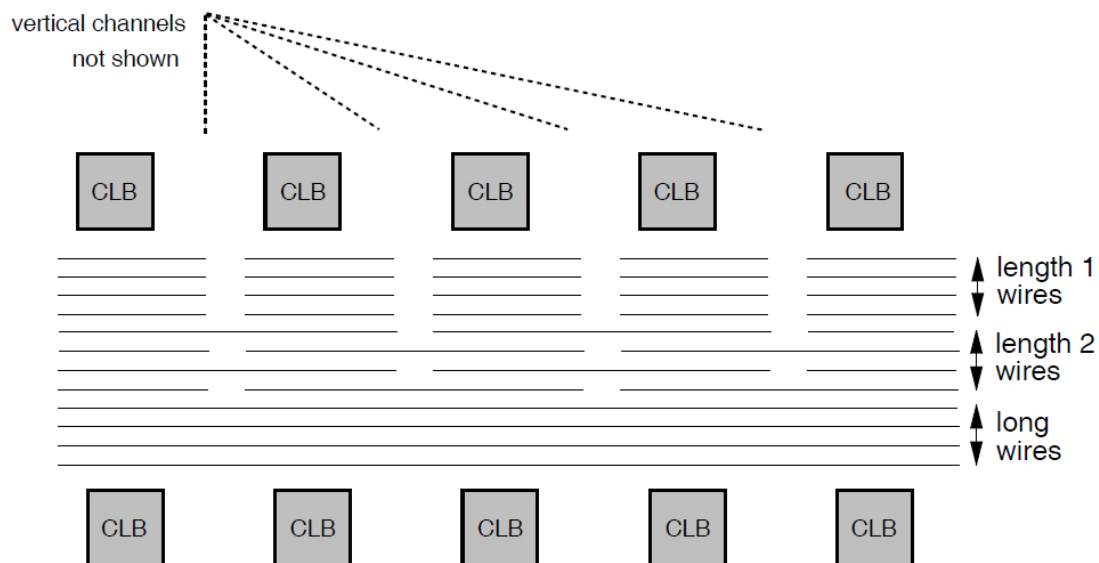
سلول منطقی CLB، FPGAهای سری XC4000 شرکت Xilinx



69

69

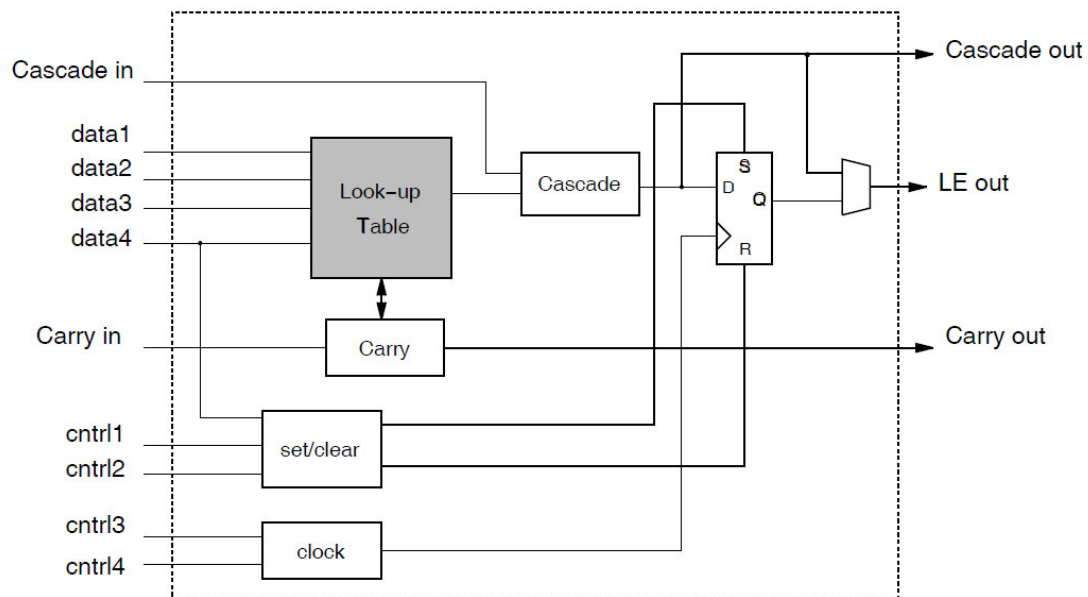
آرایه بلوک‌های منطقی CLB، FPGAهای سری XC4000



70

70

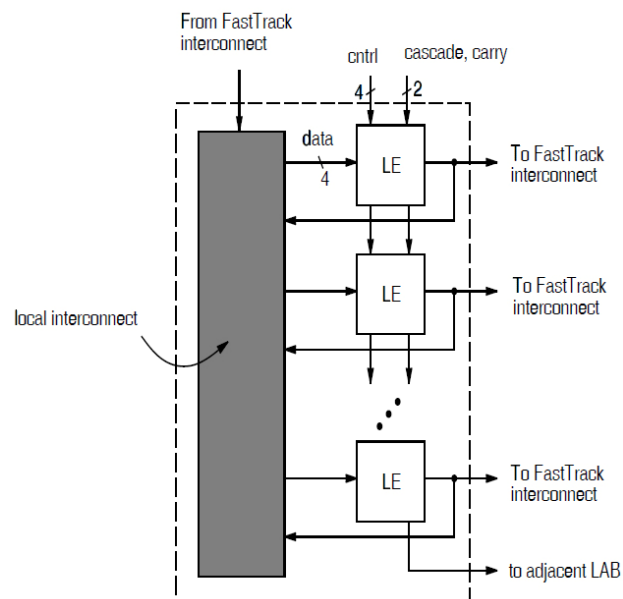
سلول یا عنصر منطقی (LE)، FPGAهای سری FLEX8000 شرکت Altera



71

71

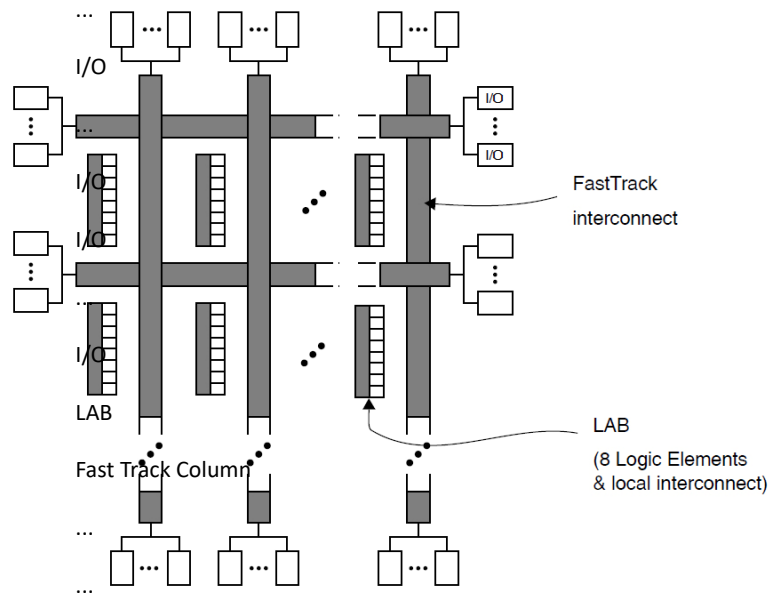
ساختار آرایه بلوک منطقی (LAB)، FPGAهای سری FLEX8000 شرکت Altera



72

72

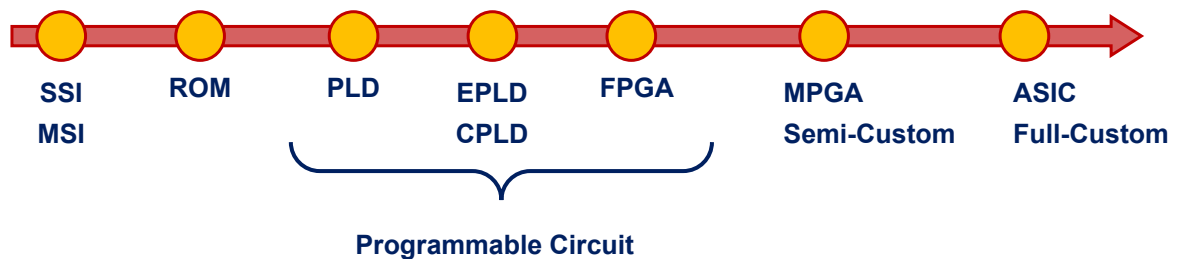
ساختار کلی FPGAهای سری FLEX8000 شرکت Altera



73

73

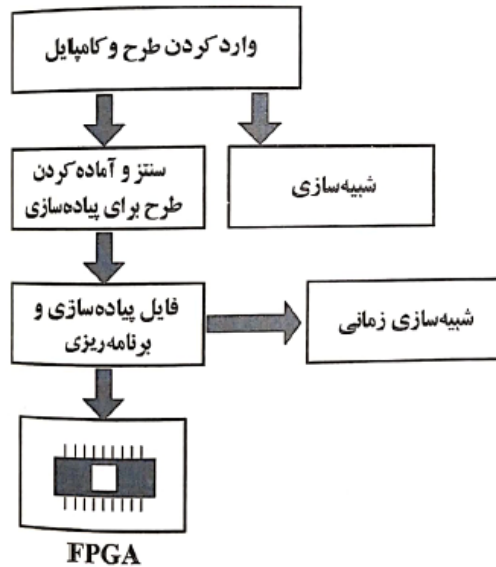
مقایسه تراشه‌ها از دیدگاه تجمع‌پذیری



74

74

روش طراحی سیستم‌های دیجیتال با FPGA



75

75

وارد کردن طرح اولیه و کامپایل

۱- ترسیم شماتیک با ادیتور گرافیک در محیط ابزار برنامه‌ریزی PPGA

۲- توصیف طرح با زبان توصیف سخت‌افزار VHDL با استفاده از ویرایشگر متن HDL

76

76

وارد کردن طرح اولیه و کامپایل

```

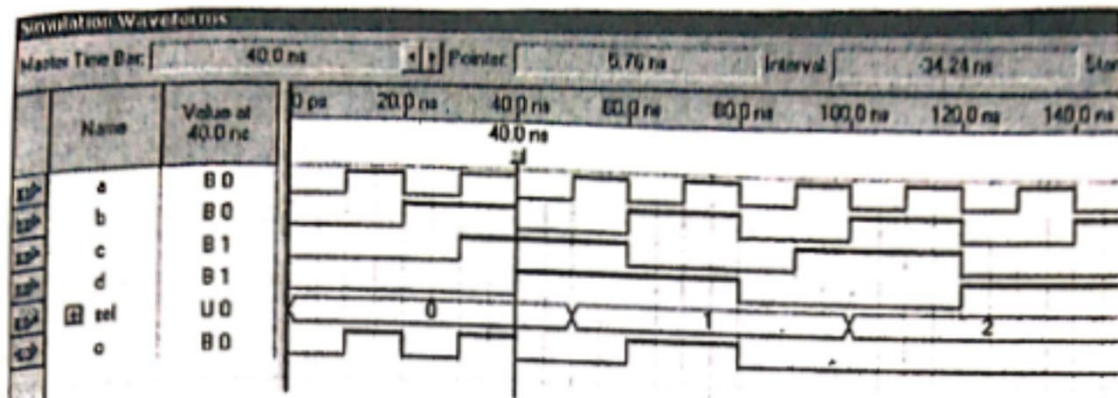
library ieee;
use ieee.std_logic_1164.all;
entity pro is
    port(a,b,c,d: in std_logic;
         sel: in std_logic_vector(1 downto 0);
         o: out std_logic);
end;
architecture MULCA of pro is
begin
    process(a,b,c,d,sel)
    begin
        case sel is
            when "00" => o <= a;
            when "01" => o <= b;
            when others => null;
        end case;
    end process;
end;

```

77

77

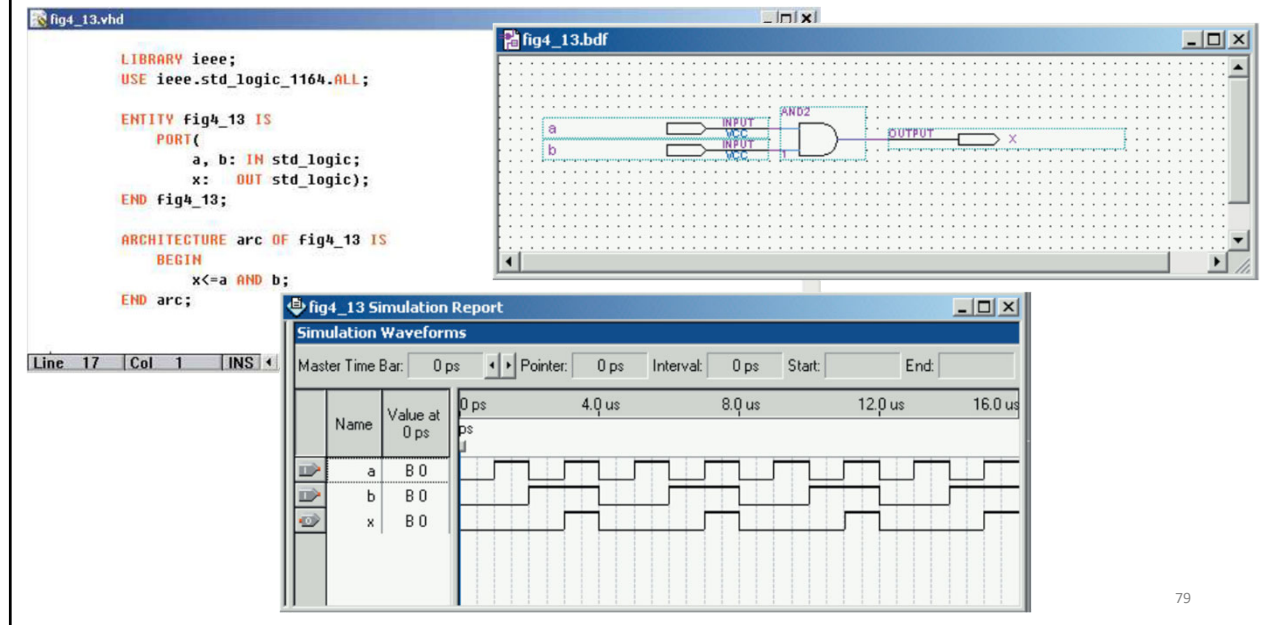
شبیه سازی



78

78

وارد کردن طرح اولیه، کامپایل و شبیه‌سازی



79

سنتز و آماده کردن طرح برای پیاده‌سازی

• سنتز یعنی تبدیل برنامه VHDL به **معادل عناصر منطقی** مانند جمع‌کننده، مولتی‌پلکسر، ثبات، دیکدر، فلیپ‌فلاپ و و اتصالات مربوطه که به آن **netlist** گفته می‌شود.

• چون طرح اولیه منطقی معمولاً به‌طور **بهینه نیست**، در ابزارهای برنامه‌ریزی FPGA الگوریتم‌هایی وجود دارد که طرح اولیه را بهینه می‌کند.

80

80

فایل پیاده‌سازی

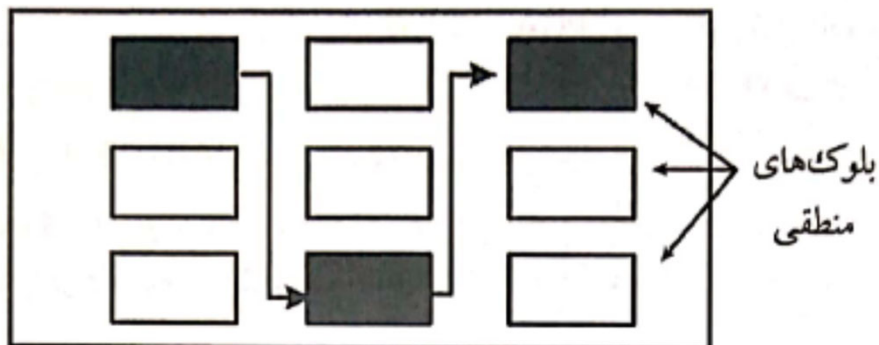
• در این مرحله نوع FPGA را مشخص می‌کنیم و با استفاده از **فایل سنتز** که شامل **بلوک‌های منطقی** و **ارتباطات** آنها می‌باشد فایل با یک سری **بیت‌های باینری** تولید می‌شود که با استفاده از آن تعدادی **سوییچ‌ها و بلوک‌های منطقی FPGA** می‌توانند برنامه‌ریزی شوند.

• در این مرحله محل **بلوک‌های منطقی مورد نیاز** در FPGA مشخص و **ارتباطات** آنها در سیلیکن FPGA نیز مشخص می‌شود.

81

81

انتخاب بلوک‌های منطقی (Placement) و ارتباط (Routing) آنها در FPGA



82

82

شبیه‌سازی زمانی (اختیاری)

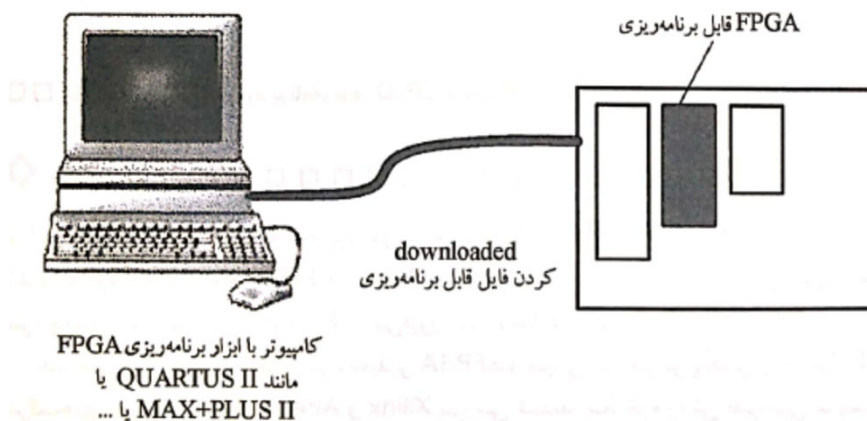
• در این مرحله مقدار تأخیر گیت‌های بلوک‌های منطقی مشخص می‌شود در نتیجه زمان‌بندی و تأخیر واقعی مدار بررسی می‌گردد.

• برخی ابزارهای برنامه‌ریزی FPGA این مرحله را مرحله **Verification** می‌نامند.

83

83

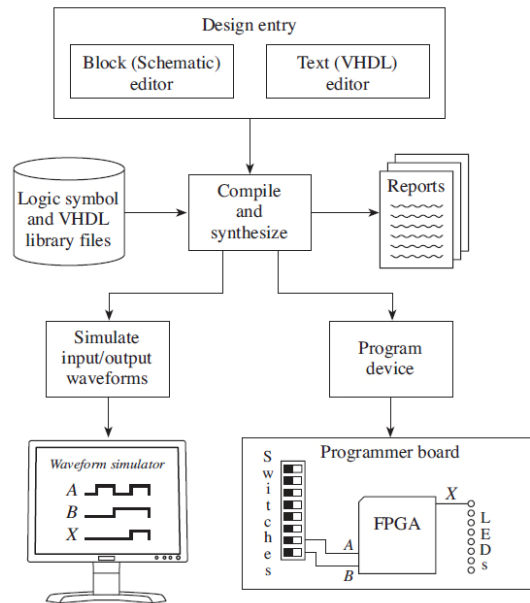
برنامه‌ریزی FPGA



84

84

روش طراحی سیستم‌های دیجیتال با FPGA



85

85